



エレメンタルハンターズ

技術紹介スライド

27卒 プログラマ志望 山口 隼(やまぐち はやと)
担当箇所:Player/NPC/InputSystem/Stage

目次

① 担当範囲

② 技術工夫による全体像

③ キャラクター共通基盤

④ 階層型StateMachine設計

⑤ コンボ/パラメーター管理

⑥ 入力システム再設計

⑦ AdapterPatternの導入

⑧ 発射リクエスト/Factory

⑨ データ駆動ステージ管理

⑩ まとめ

①担当範囲

1.Player 制御

移動/攻撃/状態管理

2.NPC 制御

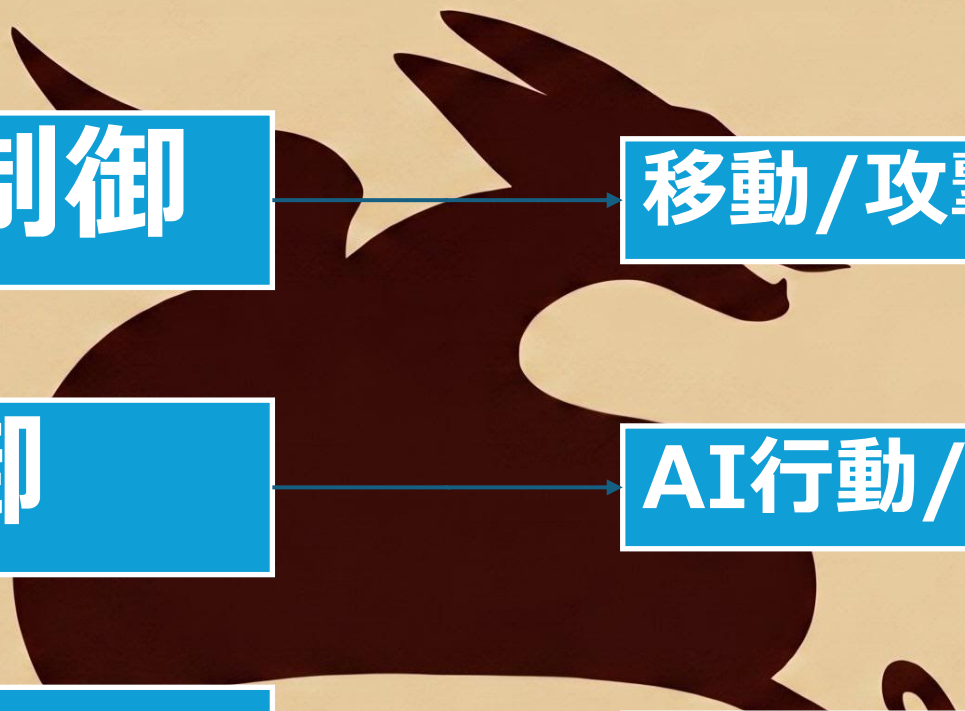
AI行動/仮想入力

3.入力設計

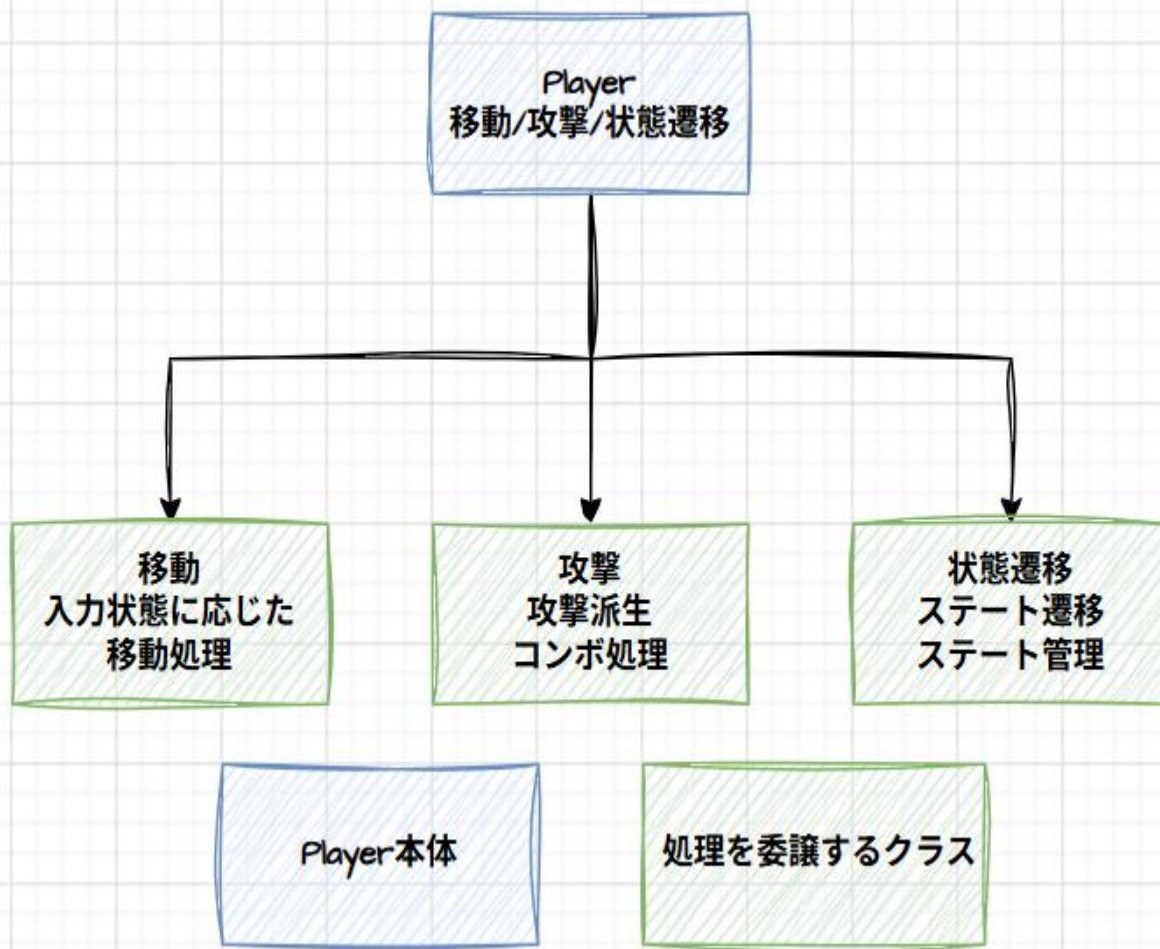
Player/NPCの入力切替

4.ステージ管理

データ駆動ステージ管理



① Player 制御

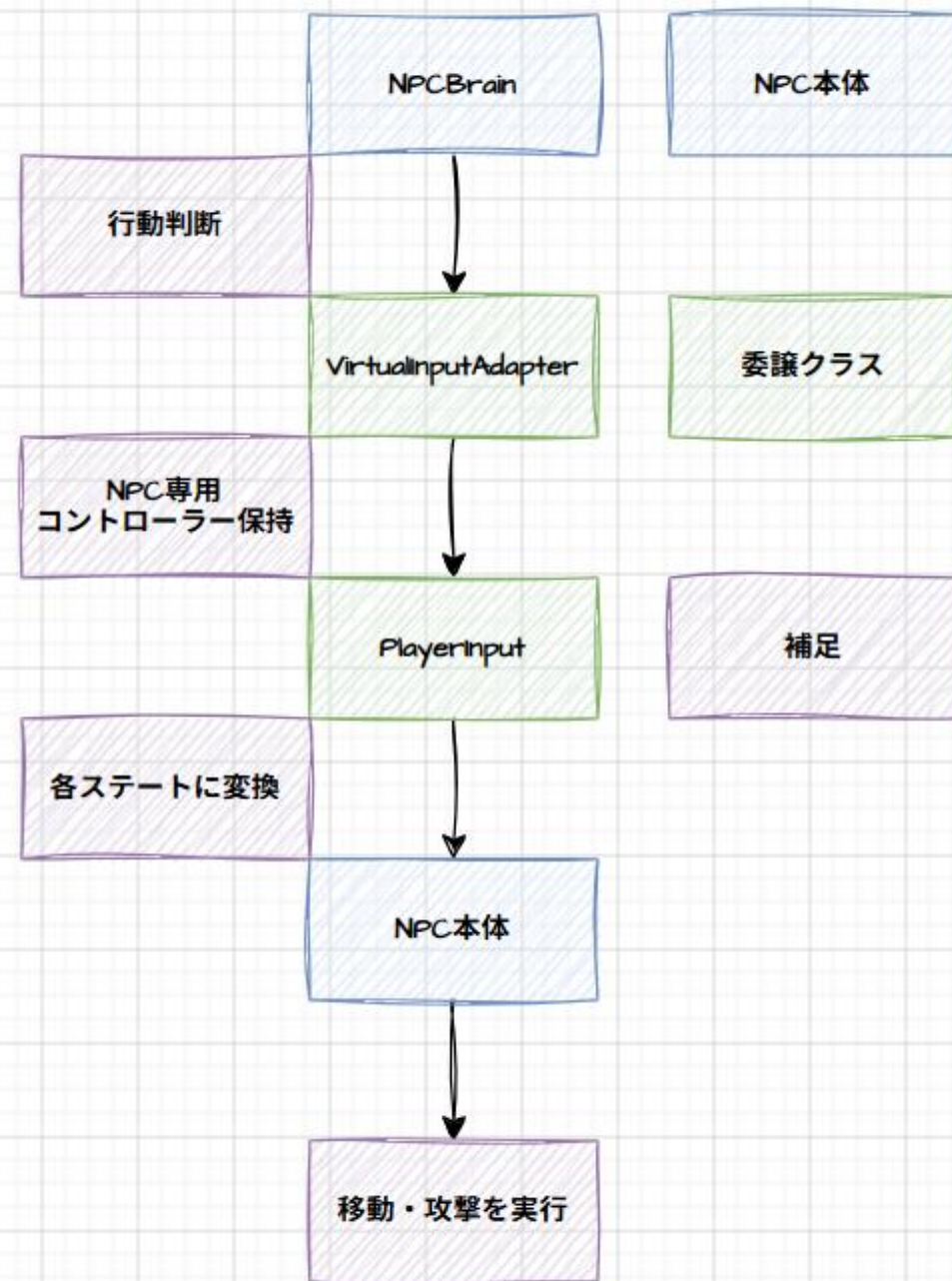


Playerクラスを

- ・ 移動処理
- ・ 攻撃処理
- ・ 状態遷移処理に分け責務分けをしました。

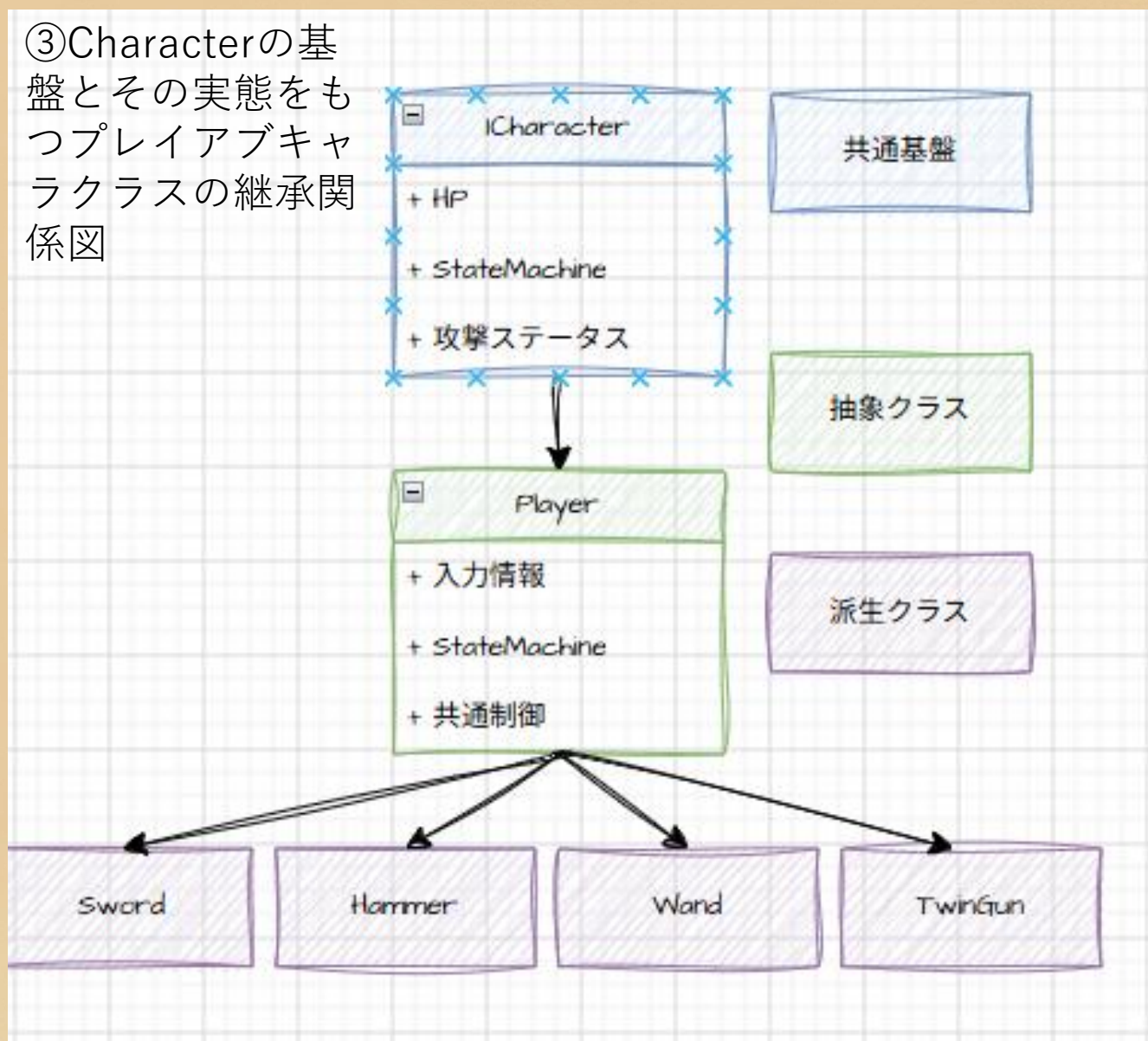
② NPC 制御

Playerの入力を物理入力、NPCを仮想入力として扱い
共通処理で制御できるように設計
しました。



③キャラクター共通基盤

③Characterの基盤とその実態をもつプレイヤーキャラクタークラスの継承関係図

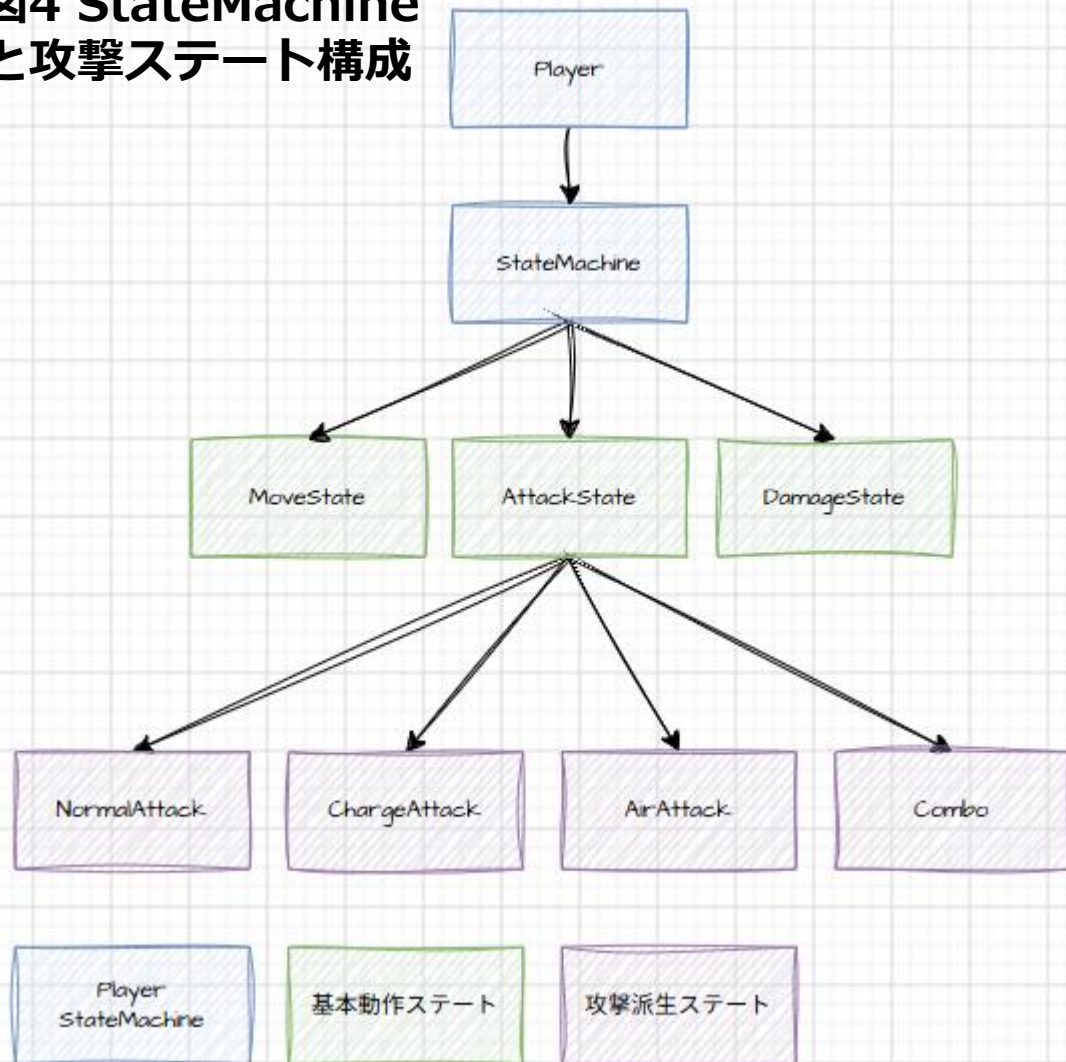


共通処理と武器固有の処理を分離しています。

結果としてキャラクターを追加する時や修正時の影響範囲を抑えやすくしました。

④階層型StateMachine設計

図4 StateMachine
と攻撃ステート構成



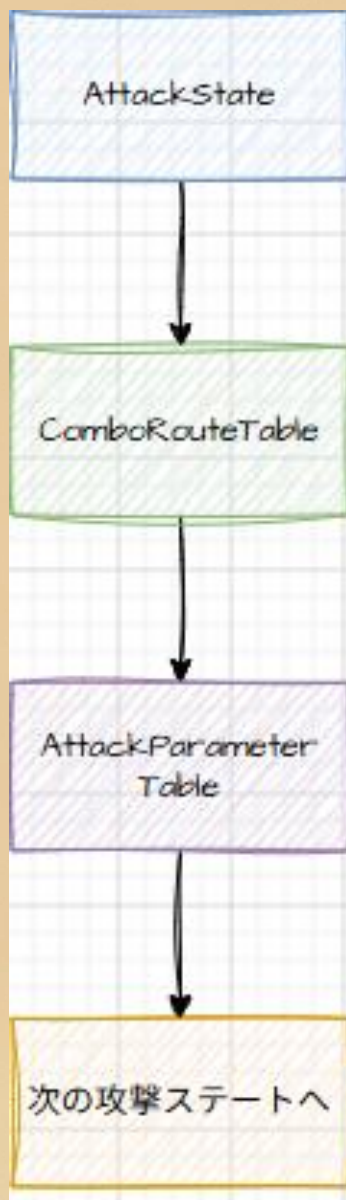
状態(ステート)ごとに処理を分岐し、Playerクラスのif文の分岐を削減しました。

実装しやすくなった点:
新規実装時はAttackStateを追加し、遷移条件を登録することで実装ができます。

関連コード:

Player/StateMachine/各AttackState
PlayerAttackBaseState

⑤コンボ/パラメーター管理



コンボ派生条件やダメージ倍率をテーブルとして定義しています。

攻撃ステートを直接変更せず、テーブル内の値を変えるだけで攻撃倍率とクリティカル倍率を調整できます。

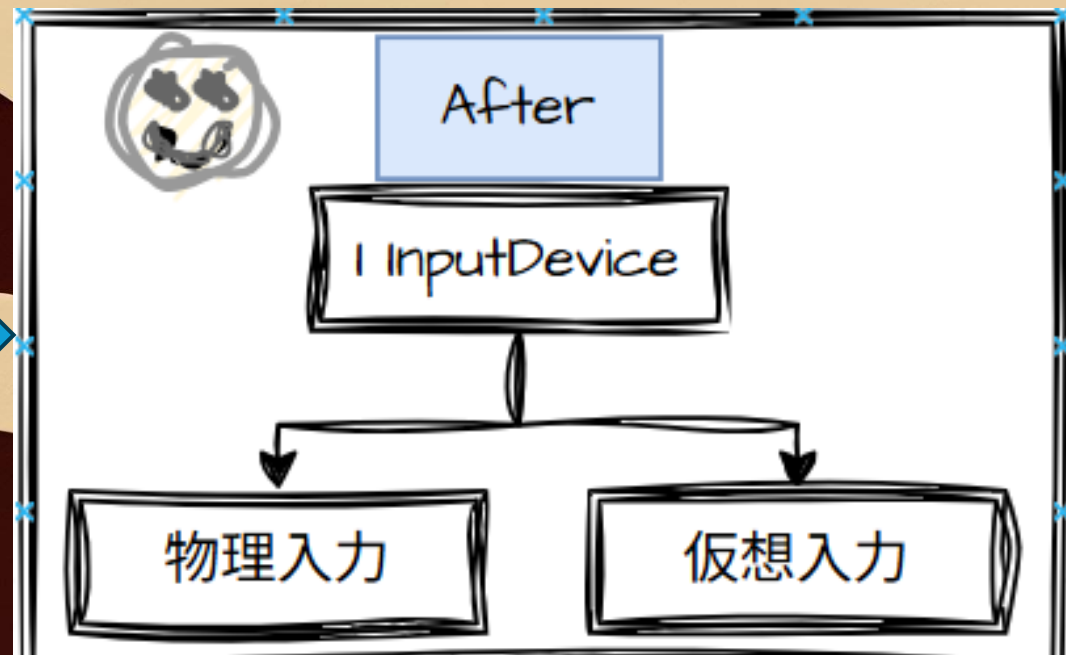
```
namespace nsApp
{
    const std::unordered_map<AttackType, AttackParameter> AttackParameterTable::m_table =
    {
        /* 通常攻撃: ダメージ倍率:等倍。クリティカル補正:なし。*/
        {AttackType::NormalAttack, {NORMAL_ATTACK_DAMAGE_RATE, 0.0f}},

        /* チャージ攻撃: ダメージ倍率:1.5倍。クリティカル補正:0.2倍。*/
        {AttackType::ChargeAttack, {CHARGE_ATTACK_DAMAGE_RATE, CHARGE_ATTACK_DAMAGE_RATE * 0.2f}},

        /* 重攻撃: ステータスはチャージ攻撃と同じ。*/
        {AttackType::HeavyAttack, {CHARGE_ATTACK_DAMAGE_RATE, CHARGE_ATTACK_DAMAGE_RATE * 0.2f}},
    };
}
```

Attack Parameter Table

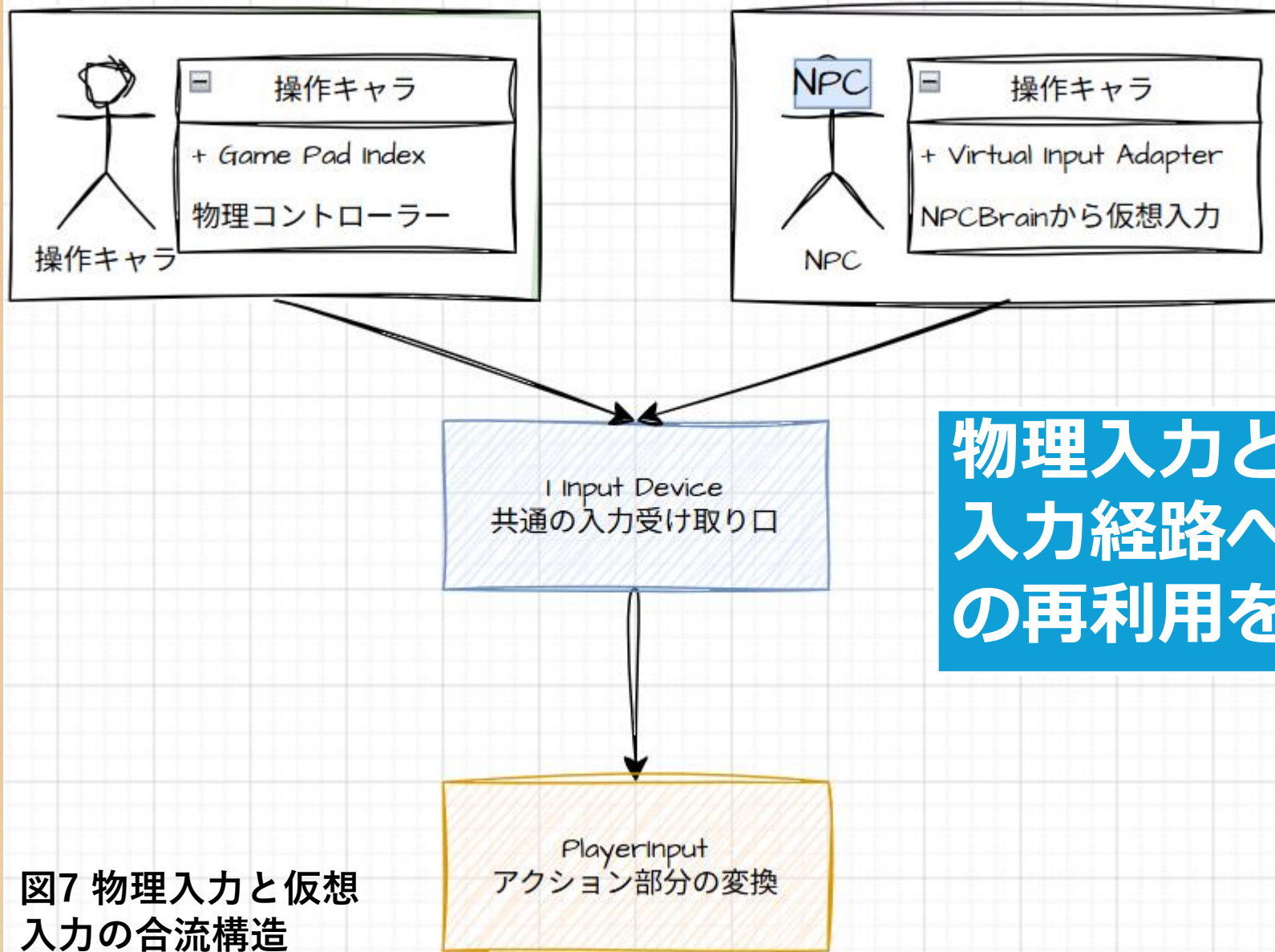
⑥ 入力システム再設計



旧設計: 1つのクラスに物理入力と仮想入力が分岐していました。

新設計: AdapterPatternで設計し、入力元を分離。これにより、`PlayerInput`はアクション部分への変換に集中できます。

⑦AdapterPatternの導入



物理入力と仮想入力を共通の入力経路へ集約しPlayerInputの再利用を可能にしました。

図7 物理入力と仮想入力の合流構造

⑧発射リクエスト/Factory

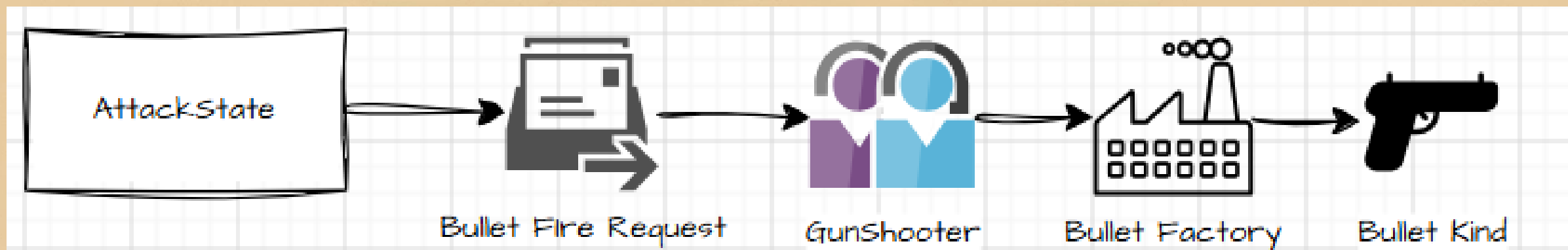


図8 発射リクエストとFactoryによる生成処理

AttackStateが生成処理を持たないよう、
発射情報Requestクラスに分離しました。

弾丸追加時はFactory側に処理を任せることで、
AttackStateの肥大化を防いでいます。

関連コード:

BulletFireRequest/GunShooter/BulletFactory/BulletParameterTable/各Bulletクラス

⑨データ駆動ステージ管理

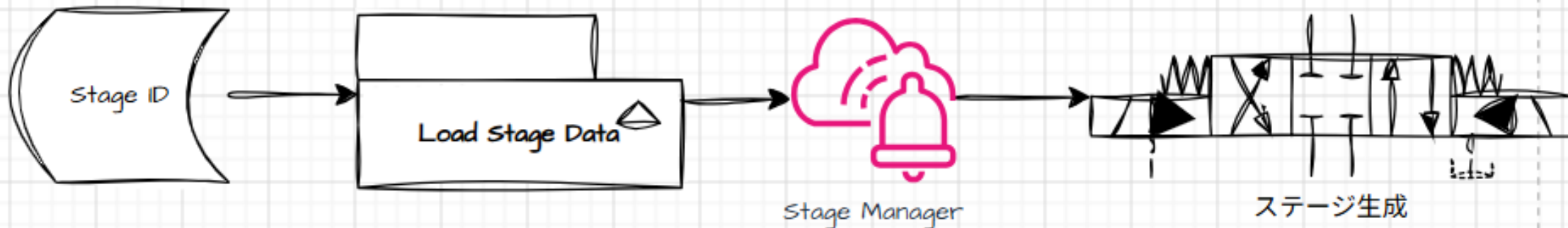


図9 データ駆動によるステージ読み込み

ステージ事の情報Stage Dataとして分離。
新しくステージが増えてもデータを登録するだけで完結し、
ステージの数だけクラスが増えることはありません。

⑩まとめ

Player / NPC / 入力情報 / 攻撃 / ステージ管理を中心に、責務分離と拡張性を意識して実装しました。

入力情報の分離、StateMachine、Factory、データ駆動を取り入れることで、機能追加や調整をしやすい構成を目指しました。

今後は、キャラクターごとの操作感やNPCの挙動をさらに調整し、協力アクションとしての遊びやすさを高めていきます。